

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING

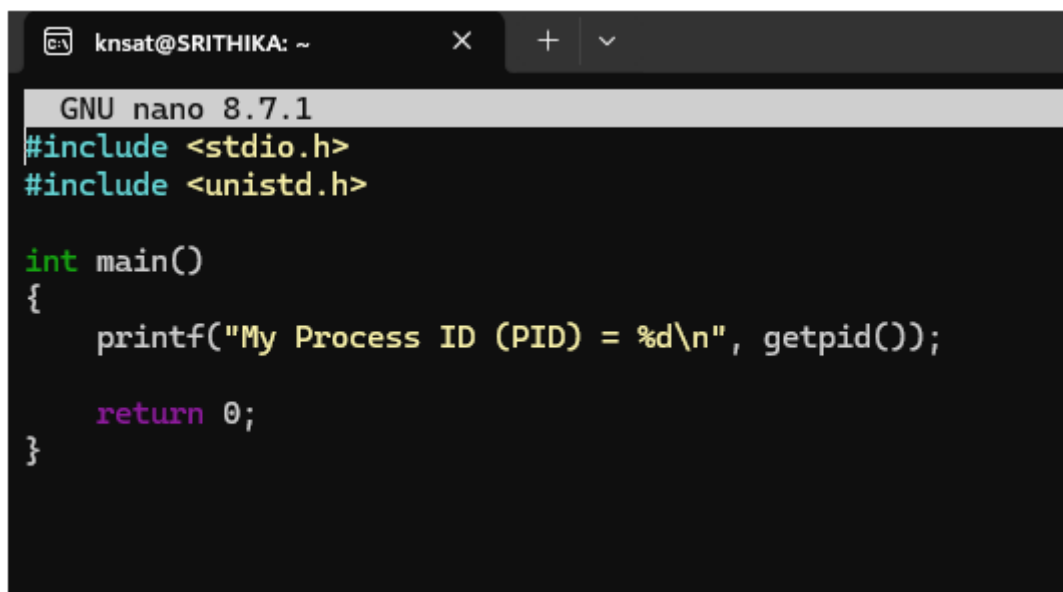
Sec:- 2

Name:- Srithika Kambhammettu

RollNo:- 2520030361

OS Practical Input and Output

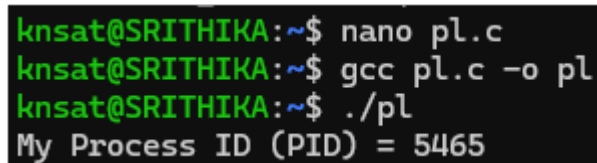
1. processid



```
knsat@SRITHIKA: ~
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("My Process ID (PID) = %d\\n", getpid());

    return 0;
}
```



```
knsat@SRITHIKA:~$ nano pl.c
knsat@SRITHIKA:~$ gcc pl.c -o pl
knsat@SRITHIKA:~$ ./pl
My Process ID (PID) = 5465
```

2. creating child using fork()

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
    }

    return 0;
}
```

```
knsat@SRITHIKA:~$ nano p2.c
knsat@SRITHIKA:~$ gcc p2.c -o p2
knsat@SRITHIKA:~$ ./p2
Parent Process
PID = 5511
Child Process
PID = 5512
```

3. Parent Waits for Child (wait())

```
knsat@SRITHIKA: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid = fork();  
  
    if(pid == 0)  
    {  
        printf("Child Process Executing...\n");  
        sleep(3);  
        printf("Child Finished\n");  
    }  
    else  
    {  
        wait(NULL);  
        printf("Parent Process Continues\n");  
    }  
  
    return 0;  
}
```

```
knsat@SRITHIKA:~$ nano p3.c  
knsat@SRITHIKA:~$ gcc p3.c -o p3  
knsat@SRITHIKA:~$ ./p3  
Child Process Executing...  
Child Finished  
Parent Process Continues
```

4. Multiple Child Processes

```
knsat@SRITHIKA: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    for(int i=1; i<=3; i++)  
    {  
        if(fork()==0)  
        {  
            printf("Child %d PID = %d\n", i, getpid());  
            return 0;  
        }  
    }  
  
    return 0;  
}
```

```
knsat@SRITHIKA:~$ nano p4.c  
knsat@SRITHIKA:~$ gcc p4.c -o p4  
knsat@SRITHIKA:~$ ./p4  
Child 1 PID = 5614  
Child 2 PID = 5615  
Child 3 PID = 5616
```

5. Execute linux command using `execvp()`

```
knsat@SRITHIKA: ~  
knsat@SRITHIKA:~$ gcc p5.c -o p5  
knsat@SRITHIKA:~$ ./p5  
Before exec()  
total 432  
drwxr-xr-x 2 knsat knsat 4096 Jul 30 04:12 LAB  
drwxr-xr-x 2 knsat knsat 4096 Jul 29 08:23 OS  
drwxr-xr-x 2 knsat knsat 4096 Jul 29 08:26 OSLab  
-rw-r--r-- 1 knsat knsat 15 Aug 12 08:33 a.txt  
-rw-r-xr-x 1 knsat knsat 16168 Aug 24 08:10 copy  
-rw-r--r-- 1 knsat knsat 515 Aug 24 08:32 copy.c  
-rw-r-xr-x 1 knsat knsat 16000 Aug 12 03:53 display  
-rw-r--r-- 1 knsat knsat 111 Aug 12 03:53 display.c  
-rw-r-xr-x 1 knsat knsat 16272 Aug 24 04:08 exec_command  
-rw-r--r-- 1 knsat knsat 439 Aug 24 03:54 exec_command.c  
-rw----- 1 knsat knsat 159 Aug 6 03:56 file.c.save  
-rw-r--r-- 1 knsat knsat 0 Jul 29 08:29 file.txt  
-rw-r--r-- 1 knsat knsat 13 Aug 12 08:31 file1.txt  
-rw-r-xr-x 1 knsat knsat 15992 Aug 13 04:04 fork  
-rw-r--r-- 1 knsat knsat 91 Aug 13 04:03 fork.c  
-rw-r-xr-x 1 knsat knsat 16000 Aug 12 04:07 forkcalling  
-rw-r--r-- 1 knsat knsat 142 Aug 12 04:07 forkcalling.c  
-rw----- 1 knsat knsat 436 Aug 6 05:18 hallo.c.save  
-rw-r-xr-x 1 knsat knsat 15992 Aug 1 05:48 hello  
-rw-r--r-- 1 knsat knsat 174 Aug 1 05:53 hello.c  
-rw-r--r-- 1 knsat knsat 54 Aug 24 08:09 input.txt  
drwxr-xr-x 2 knsat knsat 4096 Jul 30 03:41 lab  
-rw-r--r-- 1 knsat knsat 15 Aug 12 08:33 new.txt  
-rw-r-xr-x 1 knsat knsat 16168 Aug 22 04:55 orphan  
-rw-r--r-- 1 knsat knsat 239 Aug 22 04:54 orphan.c  
drwxr-xr-x 2 knsat knsat 4096 Jul 30 03:46 os  
-rw-r--r-- 1 knsat knsat 54 Aug 24 08:10 output.txt  
-rw-r-xr-x 1 knsat knsat 16080 Aug 26 05:35 p2  
-rw-r--r-- 1 knsat knsat 296 Aug 26 05:34 p2.c  
-rw-r-xr-x 1 knsat knsat 16072 Aug 26 05:37 p3  
-rw-r--r-- 1 knsat knsat 335 Aug 26 05:36 p3.c  
-rw-r-xr-x 1 knsat knsat 16032 Aug 26 05:38 p4  
-rw-r--r-- 1 knsat knsat 230 Aug 26 05:38 p4.c  
-rw-r-xr-x 1 knsat knsat 15992 Aug 26 05:40 p5  
-rw-r--r-- 1 knsat knsat 179 Aug 26 05:40 p5.c  
-rw-r-xr-x 1 knsat knsat 15992 Aug 26 05:31 pl  
-rw-r--r-- 1 knsat knsat 122 Aug 26 05:32 pl.c  
drwxr-xr-x 2 knsat knsat 4096 Jul 30 03:42 practical  
-rw-r-xr-x 1 knsat knsat 16000 Aug 25 03:59 process  
-rw-r--r-- 1 knsat knsat 112 Aug 25 03:59 process.c  
drwxr-xr-x 2 knsat knsat 4096 Jul 30 03:42 projects  
-rw-r-xr-x 1 knsat knsat 16064 Aug 12 03:42 readusername  
-rw-r--r-- 1 knsat knsat 137 Aug 12 03:41 readusername.c  
-rw-r--r-- 1 knsat knsat 32 Aug 24 08:28 sample.txt  
-rw-r-xr-x 1 knsat knsat 16192 Aug 25 04:50 simplecommandloop  
-rw-r--r-- 1 knsat knsat 609 Aug 25 04:49 simplecommandloop.c  
drwxr-xr-x 2 knsat knsat 4096 Aug 22 05:21 skill  
-rw-r-xr-x 1 knsat knsat 16208 Aug 24 15:31 states  
-rw-r--r-- 1 knsat knsat 434 Aug 24 15:33 states.c  
-rw-r--r-- 1 knsat knsat 403 Aug 24 15:31 sync.c  
-rw-r-xr-x 1 knsat knsat 16032 Aug 1 06:01 wait  
-rw-r--r-- 1 knsat knsat 209 Aug 1 06:02 wait.c  
-rw-r-xr-x 1 knsat knsat 16040 Aug 22 04:47 zombie  
-rw-r--r-- 1 knsat knsat 188 Aug 22 04:46 zombie.c  
-rw-r--r-- 1 knsat knsat 188 Aug 22 04:49 zombie.file
```

```
knsat@SRITHIKA: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    printf("Before exec()\n");  
    execvp("ls", "ls", "-l", NULL);  
    printf("This will not execute.\n");  
    return 0;  
}
```

6. Parent and Child Executing Concurrently

```
knsat@SRITHIKA: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    fork();  
  
    for(int i=1; i<=5; i++)  
    {  
        printf("PID: %d  Count: %d\n", getpid(), i);  
        sleep(1);  
    }  
  
    return 0;  
}
```

```
knsat@SRITHIKA:~$ nano p6.c  
knsat@SRITHIKA:~$ gcc p6.c -o p6  
knsat@SRITHIKA:~$ ./p6  
PID: 5707  Count: 1  
PID: 5708  Count: 1  
PID: 5707  Count: 2  
PID: 5708  Count: 2  
PID: 5707  Count: 3  
PID: 5708  Count: 3  
PID: 5707  Count: 4  
PID: 5708  Count: 4  
PID: 5707  Count: 5  
PID: 5708  Count: 5  
knsat@SRITHIKA:~$ |
```

7. Zombie Process

```
#include <stdio.h>
#include <unistd.h>
int main()
{
pid_t pid = fork();
if(pid == 0)
{
printf("Child Existing\n");
return 0;
}
else
{
sleep(15);
printf("Parent Finished\n");
}
return 0;
}
```

```
knsat@SRITHIKA:~$ nano zombie.file
knsat@SRITHIKA:~$ gcc zombie.c -o zombie
knsat@SRITHIKA:~$ ./zombie
Child Existing
Parent Finished
```

8. Orphan Process

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid > 0)
    {
        printf("Parent Exiting\n");
        return 0;
    }
    else
    {
        sleep(5);
        printf("Child PID : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }

    return 0;
}
```

```
knsat@SRITHIKA:~$ nano p7.c
knsat@SRITHIKA:~$ gcc p7.c -o p7
knsat@SRITHIKA:~$ ./p7
Parent Exiting
knsat@SRITHIKA:~$ Child PID : 5754
New Parent PID : 738
```